SIMATS ENGINEERING
ASSESSMENT TOOL 1

Analytical Problem Solving
CO1 – AT1

NAME – RAGAVI S
REG. NO – 192565027

COURSE CODE – CSA1717
COURSE NAME – ARTIFICIAL INTELLIGENCE
SLOT – D

QUESTION 1: Solution to the Water Jug Problem

> ***Problem Statement:*** *You are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring mark on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug?*
> *Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another, and no other measuring devices are available.*

1. Formal Problem Formulation

To model this problem in state space search framework, we represent the environment state as an ordered pair:

$$State = (x, y) \quad \text{where} \quad x \in \{0, 1, 2, 3, 4\} \ \text{(4-gallon jug)} \ \text{and} \ y \in \{0, 1, 2, 3\} \ \text{(3-gallon jug)}$$

- **Start State:** $(0, 0)$ — Both jugs are completely empty.
- **Goal State:** $(2, y)$ — Exactly 2 gallons in the 4-gallon jug, with any amount y in the 3-gallon jug.
- **Operators / Action Set:** Precise mathematical transitions mapping state (x, y) to (x', y').

2. Complete Set of Production Rules

Rule No.	Precondition	Action / Transition	Description
R1	$x < 4$	$(x, y) \rightarrow (4, y)$	Fill the 4-gallon jug completely from the pump.
R2	$y < 3$	$(x, y) \rightarrow (x, 3)$	Fill the 3-gallon jug completely from the pump.
R3	$x > 0$	$(x, y) \rightarrow (0, y)$	Pour all water out of the 4-gallon jug onto the ground.
R4	$y > 0$	$(x, y) \rightarrow (x, 0)$	Pour all water out of the 3-gallon jug onto the ground.
R5	$x + y \geq 4$ and $y > 0$	$(x, y) \rightarrow (4, y - (4 - x))$	Pour water from 3-gal jug into 4-gal jug until 4-gal is full.
R6	$x + y \geq 3$ and $x > 0$	$(x, y) \rightarrow (x - (3 - y), 3)$	Pour water from 4-gal jug into 3-gal jug until 3-gal is full.
R7	$x + y \leq 4$ and $y > 0$	$(x, y) \rightarrow (x + y, 0)$	Pour all water from 3-gallon jug into 4-gallon jug.
R8	$x + y \leq 3$ and $x > 0$	$(0, x + y)$	Pour all water from 4-gallon jug into 3-gallon jug.

3. Step-by-Step State Transition Solution Path

The shortest valid sequence of operations to reach the goal state is detailed below:

Step	Current State (x, y)	Rule Applied	Action Details	Resulting State Description
0	$(0, 0)$	Initial State	Start with both jugs completely empty.	4-gal: 0 gal | 3-gal: 0 gal
1	$(0, 3)$	Rule 2	Fill the 3-gallon jug to its maximum capacity from the pump.	4-gal: 0 gal | 3-gal: 3 gal
2	$(3, 0)$	Rule 7	Pour all 3 gallons from the 3-gal jug into the empty 4-gal jug.	4-gal: 3 gal | 3-gal: 0 gal
3	$(3, 3)$	Rule 2	Refill the 3-gallon jug completely from the pump.	4-gal: 3 gal | 3-gal: 3 gal
4	$(4, 2)$	Rule 5	Pour water from 3-gal jug into 4-gal jug until 4-gal is full (takes 1 gal).	4-gal: 4 gal | 3-gal: 2 gal
5	$(0, 2)$	Rule 3	Empty the full 4-gallon jug onto the ground.	4-gal: 0 gal | 3-gal: 2 gal
6	$(2, 0)$	**Rule 7**	**Pour all 2 gallons remaining in the 3-gal jug into the 4-gal jug.**	**4-gal: 2 gal | 3-gal: 0 gal (GOAL REACHED!)**

Alternative Solution Route:

Starting by filling the 4-gallon jug first: $(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3) \rightarrow (2,0)$. This path requires 6 moves (7 states) as well, proving dual-symmetry in state space.

QUESTION 2: Mars Rover Agent Specification & Architecture

Problem Statement: *Find out how Mars Rover analyzes and explores samples on Mars by answering: (i) What are the percepts for this agent? (ii) Characterize the operating environment. (iii) What are the actions the agent can take? (iv) How can one evaluate the performance of the agent? (v) What sort of agent architecture do you think is most suitable for this agent? Why?*

i. Agent Percepts (Sensory Inputs)

- **Visual & Spatial Imaging:** Stereo panoramic camera streams, Navigation Cameras (Navcams), Hazard Avoidance Cameras (Hazcams), and LIDAR point clouds for terrain mapping.
- **Kinematic & Internal Telemetry:** Wheel speed sensors, wheel torque counters, Inertial Measurement Units (IMU: accelerometers & gyroscopes for tilt/pitch/roll), and chassis strain gauges.
- **Scientific Instrumentation:** Spectrometer wavelength readings (APXS, ChemCam laser breakdown spectra), X-ray diffraction patterns, drill resistance/depth feedback, and microscopic imagery.
- **Environmental Sensors:** Atmospheric pressure, temperature sensors, wind speed indicators, and solar irradiance meters.
- **System Health:** Battery state of charge, solar panel voltage, CPU internal temperature, and memory utilization.

ii. Characterization of the Operating Environment

Environment Dimension	Classification	Detailed Engineering Justification
Observability	Partially Observable	Sensors cannot perceive beneath sub-surface soil layers or beyond line-of-sight visual terrain obstructions (e.g., hidden rocks/cliffs).
Determinism	Stochastic	Actions have uncertain outcomes; command "drive 1 meter" may result in 0.7m progress due to wheel slippage in Martian sand dunes.
Episodic / Sequential	Sequential	Current decisions (e.g., power consumption during daytime driving) directly constrain future choices (e.g., survival heating during cold Martian night).
Dynamism	Dynamic	The Martian environment changes dynamically (e.g., dust storms obscure solar panels, ambient temperatures drop) while the agent deliberates.
Continuity	Continuous	Speed, steering angle, time, thermal gradients, and power consumption are continuous physical quantities.
Agent Nature	Single / Multi-Agent	Primarily single-agent on the surface, but operates as a multi-agent system when coordinating with Mars orbiters (MRO) and Earth ground control.

iii. Agent Actions (Actuator Outputs)

- **Locomotion:** Engage wheel motors, adjust steering angle, control drive speed, execute point-turns, and engage differential lock.
- **Sample Manipulation:** Deploy robotic arm (articulated arm moves), position drill head, apply vertical drilling force, scoop soil, and deposit samples into internal cache.
- **Scientific Analysis:** Fire ChemCam laser to vaporize rock samples, open/close spectrometer shutter, run chemical analysis protocols.
- **Communication & Maintenance:** Orient High-Gain Antenna (HGA) toward Earth/Orbiter, activate internal thermal heaters, adjust solar panel angles.

iv. Performance Evaluation Metrics

- **Scientific Yield:** Total number of high-priority rock core samples collected, cached, and successfully analyzed.
- **Safety & Survival Rate:** Zero catastrophic immobilizations (e.g., avoid deep sand traps), zero mechanical failures, and maintaining positive thermal/power balance per Sol (Martian day).
- **Autonomous Navigation Efficiency:** Meters traveled per Sol without requiring human operator intervention from Earth.
- **Resource Efficiency:** Ratio of scientific payload return per Watt-hour of power consumed.

v. Suitable Agent Architecture: Hybrid Layered Architecture (Deliberative + Reactive)

Selected Architecture: A 3-Tiered Hybrid Agent Architecture comprising a *Reactive Layer* (bottom), a *Deliberative/ Planning Layer* (top), and an *Executive/Middle Layer*.

Architectural Justification:

1. **Latency Constraints:** Direct teleoperation from Earth is impossible due to signal propagation delays of 4 to 24 minutes each way. The rover must autonomously plan high-level goals.
2. **Safety-Critical Imperative (Reactive Needs):** If a wheel drops into a hidden fissure, waiting for deliberative planning could cause structural collapse. The *Reactive Layer* executes reflex behaviors (e.g., emergency stop) in milliseconds without high-level deliberation.
3. **Goal-Oriented Mission Objectives (Deliberative Needs):** Selecting sample targets, optimizing daily traversal routes, and managing power reserves require classical AI search, constraint satisfaction, and utility trade-offs executed by the *Deliberative Layer*.

QUESTION 3: Search Space Exploration for 8-Queens Problem

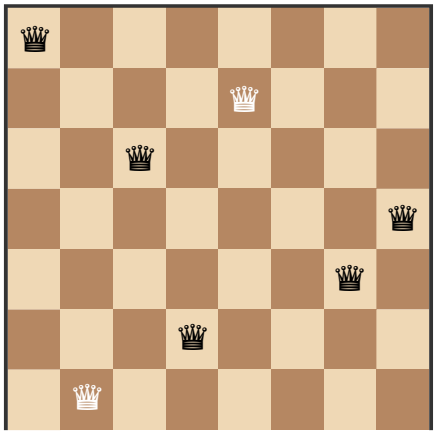
Problem Statement: *Explore the search space using search strategies and formulate the problem components for the 8-Queens problem: Place 8 queens on a chessboard such that none of the queens attack any of the others.*

1. Problem Formulation Components

- **States:** Any placement of k queens ($0 \leq k \leq 8$) on the 8×8 chessboard, or a complete configuration of 8 queens with 1 queen per column.
- **Initial State:** An empty 8×8 chessboard with zero queens placed.
- **Actions / Successor Function:** Add a queen to any empty cell in the lowest-numbered empty column such that it is not under attack by any existing queen.
- **Transition Model:** Returns the board state resulting from adding a queen at specified coordinates (row, col).
- **Goal Test:** 8 queens are placed on the board, and no two queens share the same row, column, or diagonal.
- **Path Cost:** Zero (or uniform cost per step). Only the final state configuration matters.

2. Board Configuration Illustration

Below is a visual state representation of an 8-Queens board configuration (Non-solution example with diagonal conflict between Column 1 and Column 8):



3. Search Space Size Analysis Across Formulations

Formulation Strategy	Search Space Mathematical Expression	Total State Count	Search Efficiency Analysis
1. Naive Unconstrained Placement	$64 C 8 = rac{64!}{8! imes 56!}$	**4,426,165,368** ($\approx 4.4 imes 10^9$)	Extremely inefficient. Allows multiple queens in the exact same square or row.
2. One Queen Per Column	8^8	**16,777,216** ($\approx 1.68 imes 10^7$)	Eliminates column attacks automatically. Reduces search space by a factor of 260.
3. Permutation Formulation (Distinct Rows & Cols)	$8!$	**40,320**	Eliminates both row and column attacks by definition. Search space is reduced drastically.
4. Incremental Search with Backtracking (Pruned Tree)	Pruned Depth-First Search Tree	**2,057 sequence nodes**	Checks attack conditions at each step and prunes invalid subtrees immediately. Highly optimal.

QUESTION 4: OLA Cab Booking Problem-Solving Agent & Pseudocode

> ***Problem Statement:*** *A customer wants to travel from one location to another using OLA Cab booking application. Customer selects cab type (mini, micro, sedan, shared, prime). Identify problem-solving agent type and write pseudocode.*

1. Agent Classification & Analysis

The OLA Cab booking system is best modeled as a **Goal-Based & Utility-Based Search Agent** operating in a dynamic environment.

- **Goal-Based:** The agent's explicit goal is to successfully transport the user from $Pickup\_Location$ to $Destination\_Location$.
- **Utility-Based:** Multiple valid paths and cab options exist. The agent evaluates a Multi-Attribute Utility Function to balance trade-offs between *Fare Cost*, *Driver ETA*, *Vehicle Comfort Level*, and *Driver Rating*.

2. Formal Agent Pseudocode

```
FUNCTION OLA_Cab_Booking_Agent(customer_request, spatial_graph, driver_fleet_db):
    # Step 1: Parse Customer Request Parameters
    src_loc = customer_request.pickup_location
    dest_loc = customer_request.destination_location
    req_type = customer_request.cab_type # e.g., 'Mini', 'Micro', 'Sedan', 'Prime'
```

```
    max_wait = customer_request.max_acceptable_eta

    # Step 2: Compute Shortest Route and Estimate Trip Distance/Time using A* Search
    route, trip_distance, trip_time = A_Star_Search(spatial_graph, src_loc, dest_loc)

    IF route IS NULL THEN
        RETURN "Error: No viable routing path found to destination."
    END IF

    # Step 3: Query Real-time Nearby Drivers within Radius
    candidate_drivers = driver_fleet_db.FindNearbyDrivers(src_loc, radius_km=5.0,
category=req_type)

    IF candidate_drivers IS EMPTY THEN
        RETURN "No drivers currently available for category: " + req_type
    END IF

    # Step 4: Evaluate Utility for Each Candidate Driver
    best_driver = NULL
    max_utility = -INFINITY
    surge_multiplier = CalculateSurgeFactor(src_loc, current_time)

    FOR EACH driver IN candidate_drivers DO
        pickup_eta = EstimateTravelTime(spatial_graph, driver.current_loc, src_loc)
        trip_fare = ComputeFare(trip_distance, trip_time, driver.cab_category,
surge_multiplier)

        IF pickup_eta ≤ max_wait THEN
            # Multi-Attribute Utility Function U(s)
            # Normalizing parameters: Weight_Time=0.4, Weight_Cost=0.4, Weight_Rating=0.2
            utility = (1.0 / pickup_eta) * 0.4 + (1.0 / trip_fare) * 0.4 + (driver.rating /
5.0) * 0.2

            IF utility > max_utility THEN
                max_utility = utility
                best_driver = driver
                selected_fare = trip_fare
                selected_eta = pickup_eta
            END IF
        END IF
    FOR END

    # Step 5: Execute Driver Match and Dispatch
    IF best_driver IS NOT NULL THEN
        booking_id = GenerateBooking(customer_request.user_id, best_driver.id, route,
selected_fare)
        best_driver.AssignTrip(booking_id, src_loc)
        RETURN Booking_Confirmation(booking_id, best_driver.details, selected_eta,
selected_fare)
    ELSE
        RETURN "All nearby drivers exceed maximum wait threshold. Please retry."
    END IF
END FUNCTION
```

QUESTION 5: Logistics Delivery Network using Uniform Cost Search (UCS)

> ***Problem Statement:*** *A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route has an associated travel cost (fuel + time). Determine the least-cost path from start warehouse S to destination warehouse G using Uniform Cost Search (UCS).*

1. Network Graph Structure & Parameters

We consider a standard representative weighted logistics graph with Start node S and Goal node G:

- **Nodes (Warehouses):** $V = \{S, A, B, C, D, G\}$
- **Weighted Edges (Route Travel Costs in $ / hrs):** $(S, A) = 2$, $(S, B) = 5$, $(A, C) = 3$, $(A, D) = 2$, $(B, D) = 1$, $(B, G) = 8$, $(C, G) = 4$, $(D, G) = 3$.

2. Uniform Cost Search (UCS) Trace Execution Table

UCS expands the node with the lowest cumulative path cost $g(n)$ using a Priority Queue (Min-Heap).

Iteration	Node Popped (Expanded)	Cost $g(n)$	Priority Queue Queue State *(Node, Cost, Path)*	Explored Set
0	—	0	*[(S, 0, [S])]*	*{}*
1	**S**	0	*[(A, 2, [S,A]), (B, 5, [S,B])]*	*{S}*
2	**A**	2	*[(D, 4, [S,A,D]), (B, 5, [S,B]), (C, 5, [S,A,C])]*	*{S, A}*
3	**D**	4	*[(B, 5, [S,B]), (C, 5, [S,A,C]), (G, 7, [S,A,D,G])]*	*{S, A, D}*
4	**B**	5	*[(C, 5, [S,A,C]), (G, 7, [S,A,D,G]), (G, 13, [S,B,G])]*	*{S, A, D, B}*
5	**C**	5	*[(G, 7, [S,A,D,G]), (G, 9, [S,A,C,G])]*	*{S, A, D, B, C}*
6	**G (Goal)**	**7**	**Goal Popped! Optimal Path Confirmed.**	*{S, A, D, B, C, G}*

3. Results & Engineering Interpretation

Optimal Solution Summary:

- **Least-Cost Path:** $S \rightarrow A \rightarrow D \rightarrow G$
- **Minimum Total Delivery Cost:** $g(G) = 7$ units (Fuel + Overhead Cost).
- **Completeness & Optimality Proof:** UCS is complete and optimal provided step costs are strictly positive ($c(e) \geq \varepsilon > 0$). Since all edge costs in the network are non-negative, expanding nodes in monotonically increasing order of $g(n)$ guarantees that when Goal node G is popped from the Priority Queue, no cheaper path exists.

- **Logistics Engineering Relevance:** In real-world automated logistics networks, UCS minimizes total transport fuel costs and carbon footprint without requiring complex heuristic evaluation functions, making it a robust foundational algorithm for fleet routing.